Systems programming

COM 312: BSc Computer science

Lesson 6: updated notes.

Topics area coverage.

UNIX Thread management and synchronization

- Posix standards and its sub group
- What is thread? Importance of threads.
- Threading Concept in systems programming
- Thread ID calls
- Mutex and semaphore in systems administration

Posix standards and its sub group

Posix standards and its sub group

- **What is posix standards? "**Portability operating system interface**"** is the name of a family related standards specified by IEEE to define the application programming interface (API), along with shell utilities interface for the software compatible with variants of the UNIX operating system.

- Because many versions of UNIX exist today and each of them provides its own set of API functions, it is difficult for system developers to create applications that can be easily ported to different versions of UNIX.

- Some of **the subgroups** of POSIX are POSIX.1, POSIX.1b & POSIX.1c are concerned with the development of set of standards for system developers.

1. **POSIX. 1**
   - This committee proposes a standard for a base operating system API; this standard specifies APIs for the manipulating of files and processes.
   - It is formally known as IEEE standard 1003.1- 1990 and it was also adopted by the ISO as the international standard ISO/IEC 9945: 1: 1990.

2. POSIX.1b

- This committee proposes a set of standard APIs for a real time OS interface; these include IPC (interposes communication)
- This standard is formally known as IEEE standard 10034-1993.

3. Posix.1c

- This standard specifies multi-threaded programming interface. This is the newest POSIX standard.
- These standards are proposed for a generic OS that is not necessary be UNIX system.
- E.g VMS from Digital Equipment corporation, OS/2 from IBM, & Windows NT from Microsoft corporation are POSIX-compliant, yet they are not UNIX systems.
- To ensure a user program conforms to POSIX.1 standard the user should either define the manifested constant_ POSIX_SOURCE at the beginning of each source module of the program (before inclusion of any header) as; # define_POSIX_SOURCE OR specify the –D_ POSIX_SOURCE option to a C++ compiler(CC) in a compilation; % CC –D_ POSIX_SOURCE *.C
- POSIX.1b defines different manifested constant to check conformance of user program to that standard. The new macro is POSIX_C_SOURCE and its value indicates POSIX version to which a user program conforms. Its value can be:

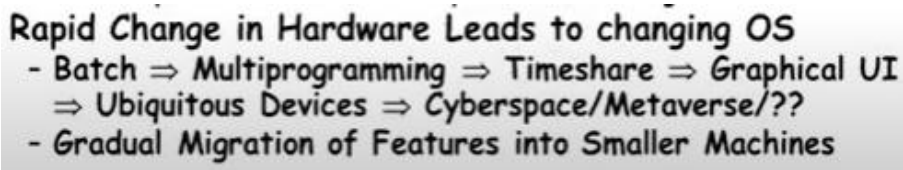
Threading Concept in programming.

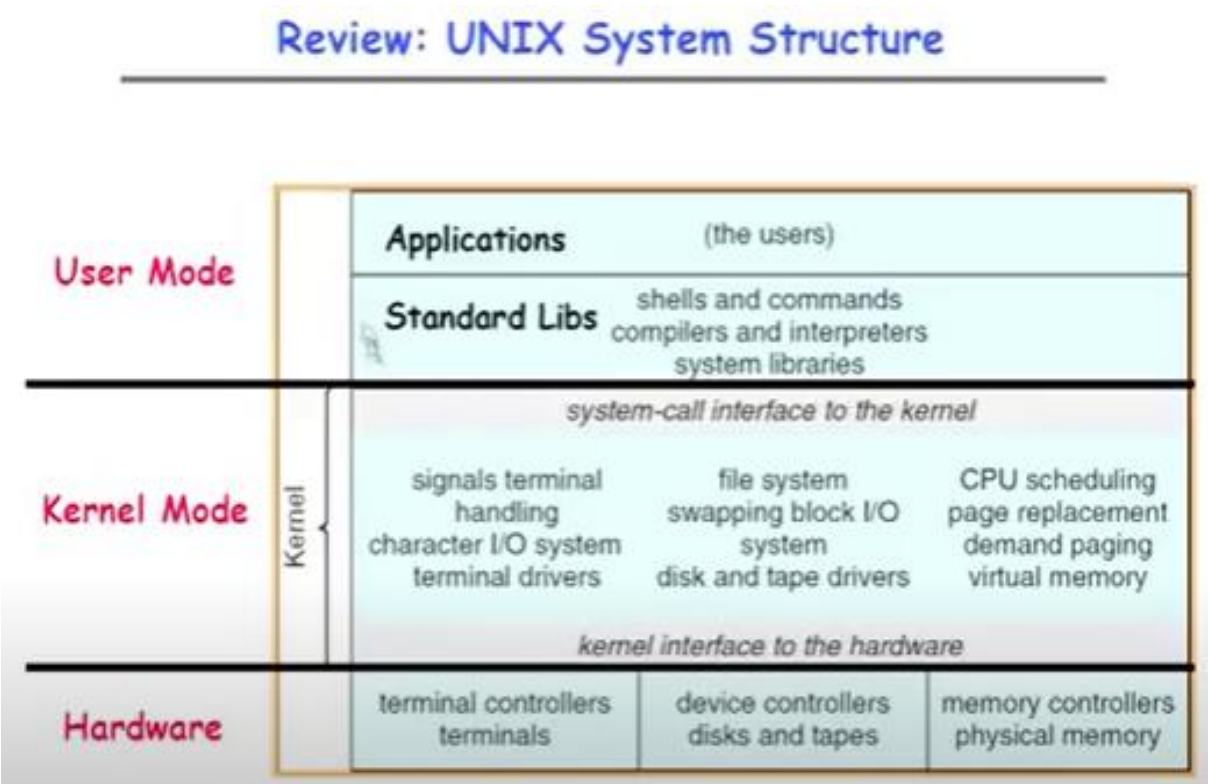
- **Threads** represent a software approach to improving performance of operating system through parallel computing by reducing the overhead thread is equivalent to a classical process.

- in modern programming **whenever a process has multiple tasks to perform independently of the others**.

What is a Thread?

- A thread is a path of execution within a process. A process can contain multiple threads.
  [Why study Threads in Computer science and Programming]





Why Multithreading?

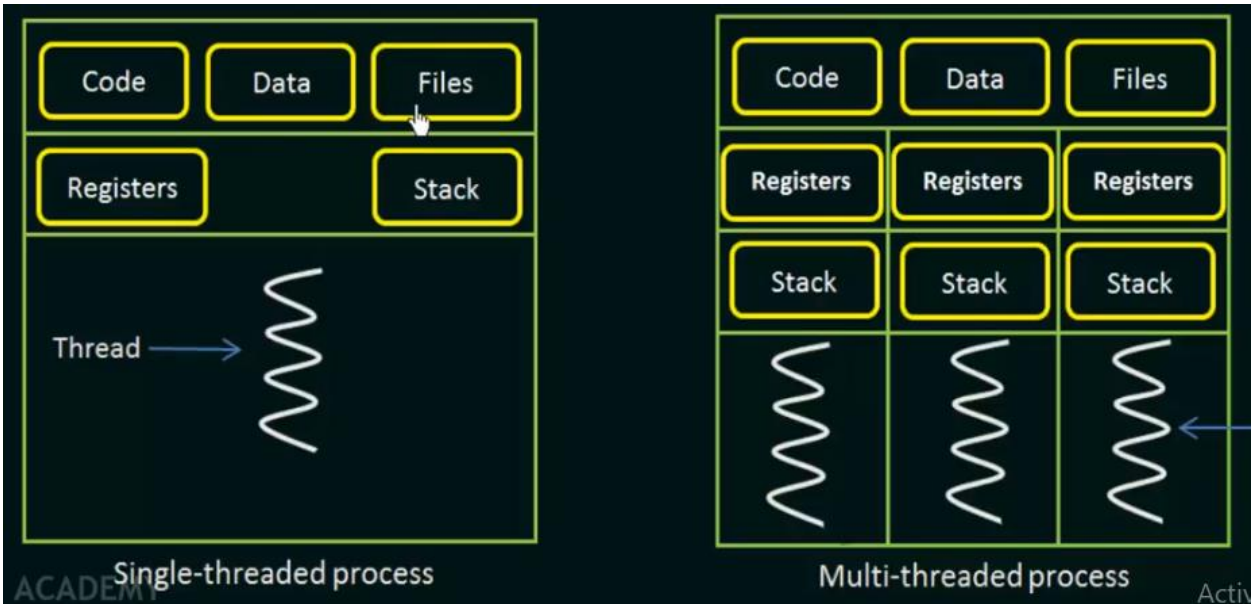
- A thread is also known as lightweight process.
- The idea is to achieve parallelism by dividing a process into multiple threads. **For example**, in a browser, multiple tabs can be different threads. MS Word uses multiple threads: *one thread to format the text*, another thread *to process inputs*, etc. More advantages of multithreading are discussed below

Process vs Thread?

- The primary difference is that

- **threads** within the same process *run in a shared memory space,*
- while **processes** run *in separate memory spaces.*

- Threads are not independent of one another like processes are, and as a result threads share with other threads their code section, data section, and OS resources (like open files and signals).
- **But, like process,** a thread has its own program counter (PC), register set, and stack space.



This is the high-level definition of threads, let's take a slightly deeper look into how threads are stored in memory.

Thread Control Block

Thread ID
Thread state
CPU information : Program counter Register contents
Thread priority
Pointer to process that created this thread
Pointer(s) to other thread(s) that were created by this thread

Thread ID – A unique identifier that gets generated by the operating system when a thread gets created.

Thread States – State of the thread that changes as the thread progresses throughout the system.

CPU information – This includes everything that the OS needs to know about the thread such as Program Count and Register Contents.

Thread Priority – It indicates the weight (or priority) of the thread over other threads which helps the thread scheduler to determine which thread should be selected next from the READY queue.

Process Pointer – A pointer to the process that started this thread.

Thread Pointer(s) – A pointer or list of pointers to other threads that were created by the current thread. Now that we know

How do multiple threads speed up my code?

Let's take a web server as an example.

- If the server created a separate process for every HTTP request, we would wait forever for our page to load. The reason is that creating a new process is an expensive procedure for the operating system.

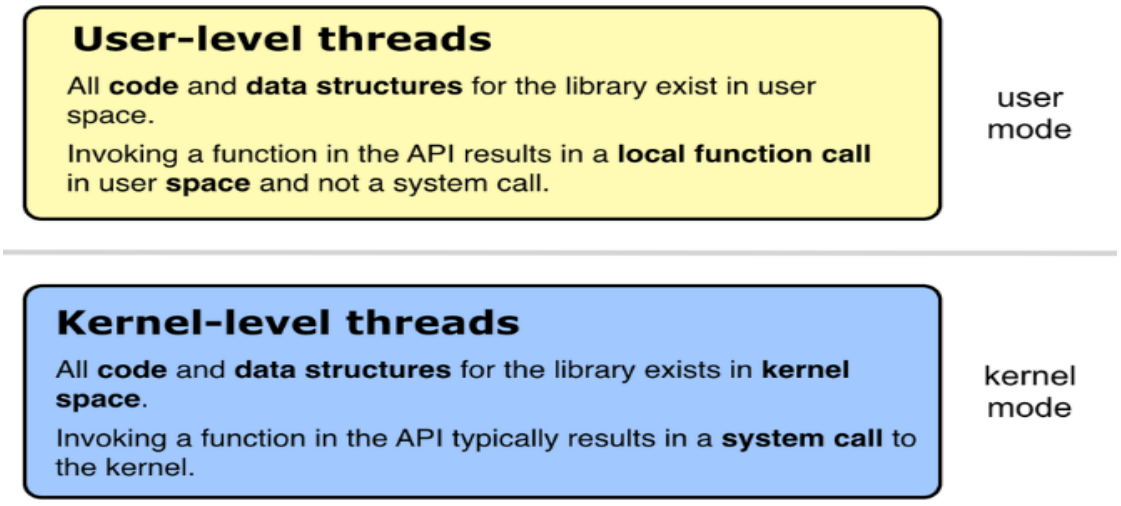
- Instead of creating a new process, it would be more efficient to create a new thread.

- But sometimes this wouldn't speed up your application. If your lacking program uses a lot of computing power, then utilizing multiple processes would be more beneficial.

- On the other hand, if you're facing an Input-Ouput (IO) problem, then multithreading will help you.

Next, let's look at different types of threads and their relationships to each other.

Thread Types



What are Pthreads/posix threads?

- Pthreads is a C/C++ library used to manage threads that are based on the POSIX standard.

The methods in the Pthreads library can be categorized into four groups:

1. **Thread Management –** Routines that work directly on threads - creating, detaching, joining, etc. They also include functions to set/query thread attributes (joinable, scheduling etc.)
2. **Mutexes –** Routines that deal with synchronization, called a "mutex", which is an abbreviation for "mutual exclusion". Mutex functions provide for creating, destroying, locking and unlocking mutexes. These are supplemented by mutex attribute functions that set or modify attributes associated with mutexes.
3. **Condition variables –** Routines that address communications between threads that share a mutex. Based upon programmer specified conditions. This group includes functions to create, destroy, wait and signal based

upon specified variable values. Functions to set/query condition variable attributes are also included.

4. **Synchronization –** Routines that manage read/write locks and barriers.

Identifying a thread

Each thread identified by an ID, which is known as Thread ID. Thread ID is quite different from Process ID. *A Thread ID is unique in the current process*, while *a Process ID is unique across the system.*

Thread ID is represented by type **pthread_t**

Header file(s)

The header file which needs to be included to access thread functions

```
#include<pthread.h>
```

Creating a Thread (pthread_create)

pthread_create is the function of pthread.h header file, which is used to create a thread. The syntax and parameters details are given as follows:

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
   void *(*start_routine) (void *), void *arg);
```

pthread_t *thread

It is the pointer to a pthread_t variable which is used to store thread id of new created thread.

const pthread_attr_t *attr

It is the pointer to a thread attribute object which is used to set thread attributes, NULL can be used to create a thread with default arguments.

void *(*start_routine) (void *)

It is the pointer to a thread function; this function contains the code segment which is executed by the thread.

void *arg

It is the thread functions argument of the type void*, you can pass what is necessary for the function using this parameter.

int (return type)

If thread created successfully, return value will be 0 (Zero) otherwise pthread_create will return an error number of type integer.

Consider the example using thread

```
#include <stdio.h>
#include <unistd.h>  //Header file for sleep(). man 3 sleep for details.
#include <pthread.h>

void *myThreadFun()
{

    sleep(5);
    printf("Printing Thread \n");
    return NULL;
}

int main()
{
    pthread_t thread_id;//thread declaration of type 'pthread_t'
    printf("Before Thread\n");
    pthread_create(&thread_id, NULL, myThreadFun, NULL); //thread is created
    pthread_join(thread_id, NULL); //process waits for thread to finish
    printf("After Thread-common message\n");

}
```

Output of P Thread

```
File  Edit  View  Search  Terminal  Help
[centos@localhost osprogs]$ gedit thread0.c
[centos@localhost osprogs]$ gedit thread0.c
[centos@localhost osprogs]$ gcc thread0.c -lpthread
[centos@localhost osprogs]$ ./a.out
Before Thread
Printing Thread
After Thread-common message
[centos@localhost osprogs]$
```

What is process synchronization in operating system?

- **Synchronization** - *is the way by which processes that share the same memory space are managed in an operating system.*
- When more than two processes have access to the same data or resources at the same time it can cause data inconsistency so to remove this data inconsistency processes should be synchronized with each other.

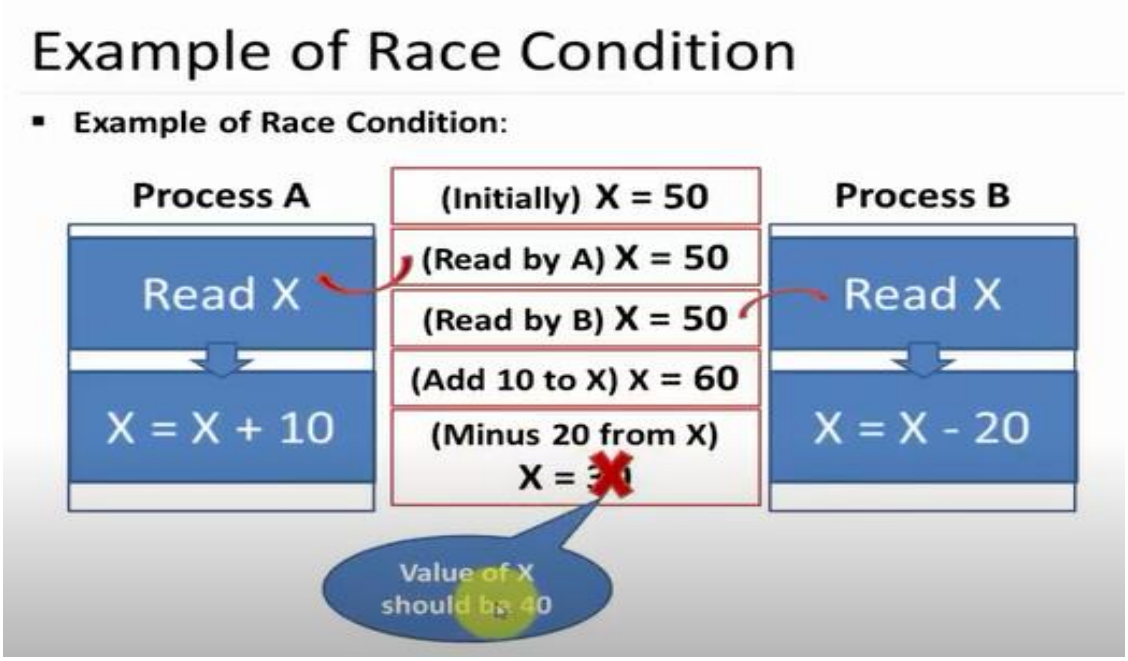


In the above picture, we take an example of a bank account that has a current balance of 500 and there are two users which have access to that account**. User 1 and User 2 both are trying to access the balance.** If process 1 is for **withdrawal** and process 2 is for **checking the balance** both will occur at the same time, then user 1 might get the wrong current balance*. To avoid this kind of data inconsistency process synchronization in os is very helpful.*

Two important concepts that are related to process synchronization i.e. Race Condition and Critical Section

Race condition

- A **race condition** is a condition when there are many processes and every process shares the data with each other and accessing the data concurrently, and the output of execution depends on a particular sequence in which they share the data and access.



Critical Section

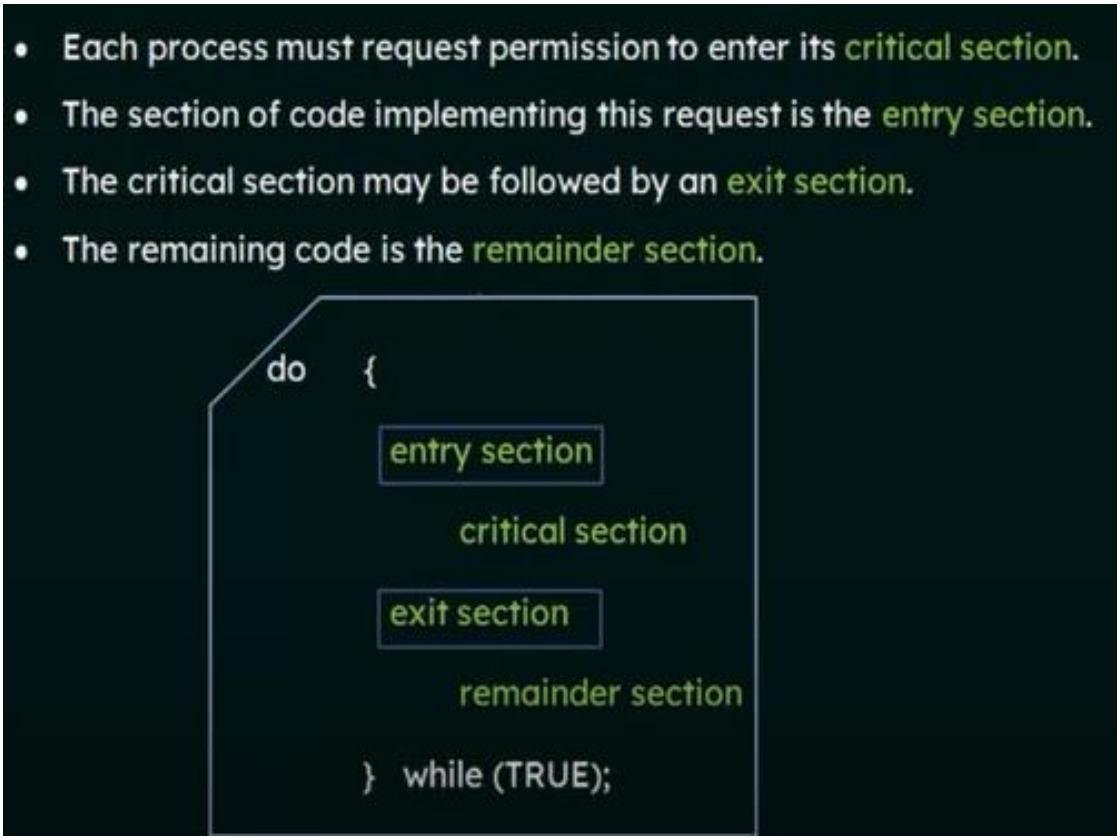
- The term "**critical section**" refers to a code segment that is accessed by several programs. The critical section includes shared variables or resources that must be synchronized in order to ensure data consistency.

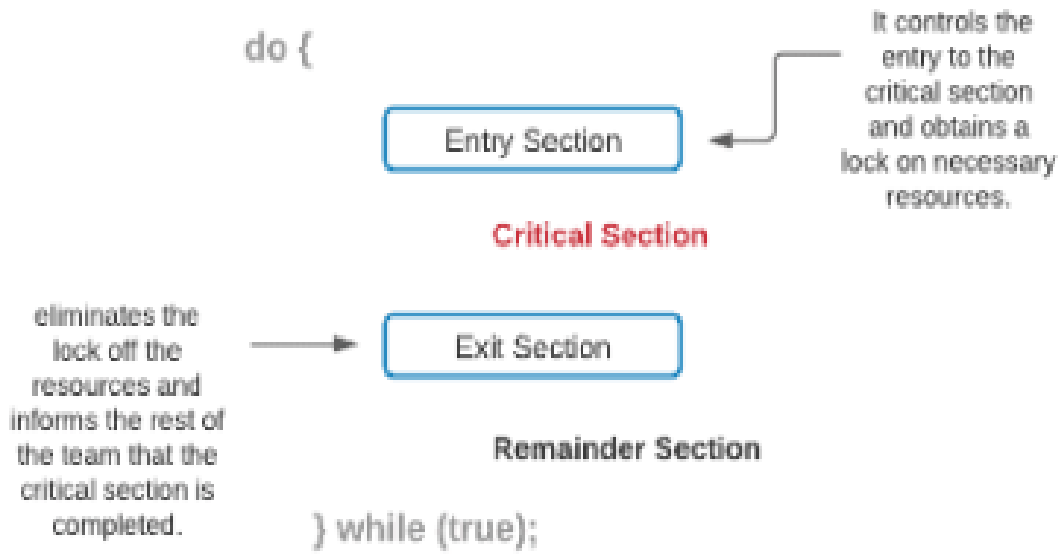
- **Critical Section** *is the part of a program which tries to access shared resources.* That resource may be any resource in a computer like a memory location, Data structure, CPU or any IO device.

- The critical section cannot be executed by more than one process at the same time; operating system faces the difficulties in allowing and disallowing the processes from entering the critical section.

- The **critical section problem** is used to design a set of protocols which can ensure that the Race condition among the processes will never arise.

General structure of a typical process





Solutions to the critical section problem

Any solution to the critical section problem must satisfy the following requirements:

1. **Mutual exclusion:** When one process is executing in its critical section, no other process is allowed to execute in its critical section.
2. **Progress**: When no process is executing in its critical section, and there exists a process that wishes to enter its critical section, it should not have to wait indefinitely to enter it.
3. **Bounded waiting:** There must be a bound on the number of times a process is allowed to execute in its critical section, after another process has requested to enter its critical section and before that request is accepted.

Most solutions to the critical section problem utilize locks implemented on software. The solutions include:

1. **test_and_set:** Uses a shared boolean variable lock and the test_and_set instruction that executes atomically and sets the passed parameter value to true.

2. compare_and_swap: Same as test_and_set, except that the passed parameter value is set to true if it is equal to expected in the parameter of the compare_and_swap instruction.

3. **Mutex locks**: Software method that provides the acquire () and release () functions that execute atomically.

4. **Semaphores:** More sophisticated methods that utilize the wait () and signal () operations that execute atomically on Semaphore S, which is an integer variable.

5. **Condition variables:** Utilizes a queue of processes that are waiting to enter the critical section.

Uses of Critical section

1. **Critical sections in data structures**

The read-write conflicting variables are divided between threads, with a copy in each thread. Data structures such as linked lists, trees, and hash tables have connected data variables that cannot be divided across threads, making parallelism difficult to implement.

To avoid this, one technique is to keep the complete data structure in a critical section and handle just one action at a time. Another option is to lock the critical section node in use so that other actions do not utilize the same node. As a result, using the critical section assures that the code produces the intended results.

2. **Critical sections in computer networking**

In computer networking, critical sections are also required. Data may not arrive in an ordered fashion when it reaches network sockets. Let's pretend that program X on the computer needs to take data from the socket, reorganize it, and check for any errors. No other application should access the same socket for that data while this program is working on it. As a result, the socket's data is secured by a critical section, allowing program X to use it exclusively.

3. **Kernel-level critical sections**

Critical section often blocks thread and process migration across processors, as well as interruptions and other processes and threads from preempting processes and threads. Nesting is common in critical parts. Multiple essential parts can be accessed and exited at little cost because of nesting.

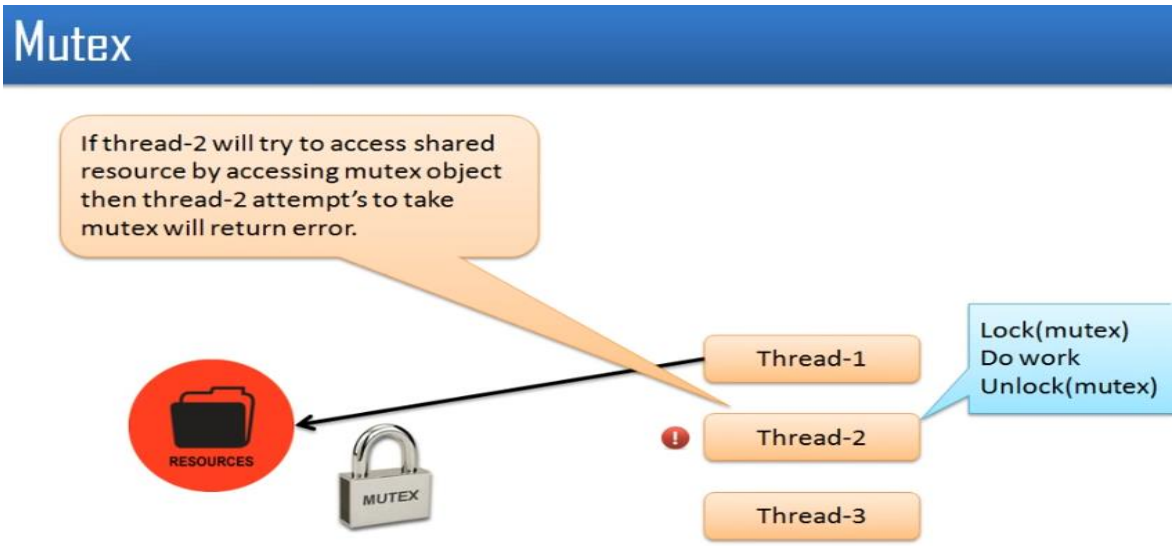
Because the critical sections may only operate on the processor into which they are inserted, synchronization is only needed inside the executing processor. This enables important portions to be entered and exited at virtually no cost. There is no need for inter-processor synchronization. All that is required is instruction stream synchronization. Most CPUs offer the necessary synchronization simply by stopping the current execution state.

4. The usage of critical sections as a long-term locking primitive is not recommended. The software lockout problem is based on kernel-level critical sections.

Mutex and Semaphore

Mutex

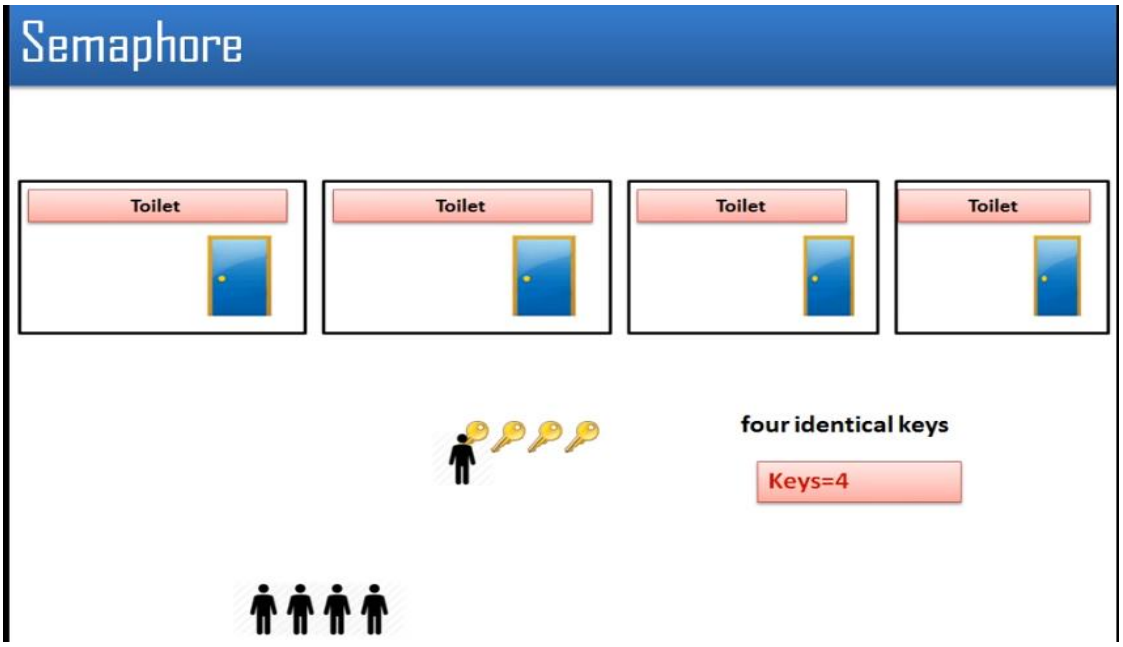


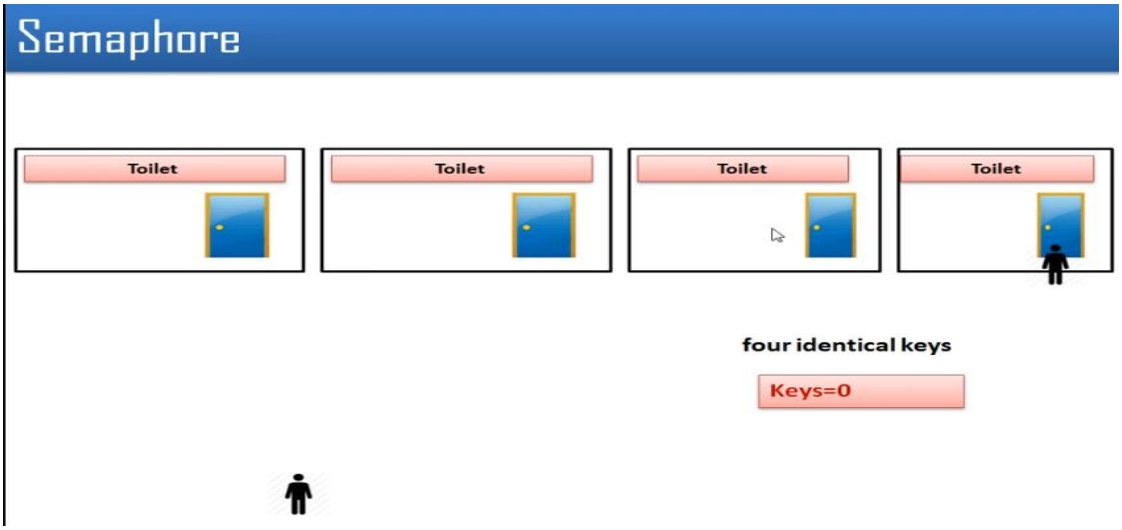


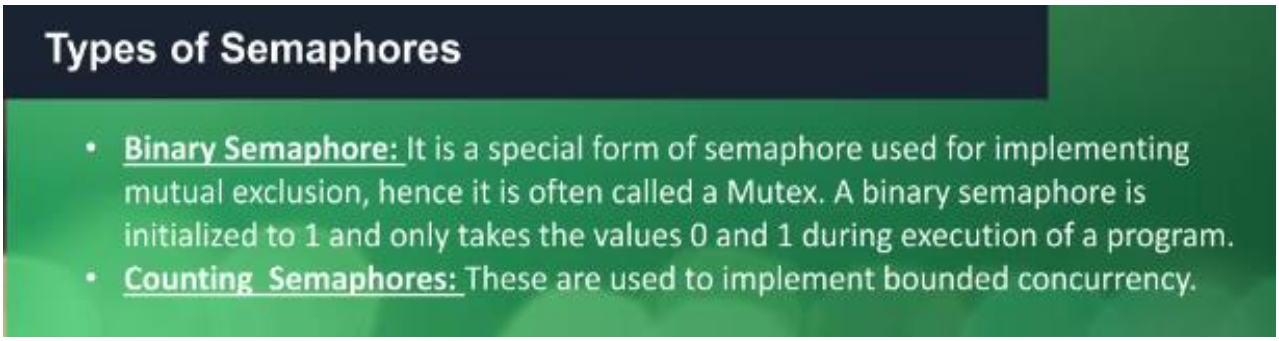
Semaphore

Semaphore is a non-negative variable that is shared across threads. It's a different algorithm or solution to the problem of the critical section. It's a thread that's waiting for a semaphore, which can be signaled by another thread.

For process synchronization, it employs two atomic operations: 1) wait() and 2) signal().







PRACTICE PROBLEMS BASED ON BINARY SEMAPHORES IN OS-

Problem-01:

Each process Pi, i = 1, 2, …, 9 is coded as follows-

repeat

P(mutex)

{ Critical Section }

V(mutex)

forever

QUIZ

The code for P10 is identical except that it uses V(mutex) in place of P(mutex). What is the largest number of processes that can be inside the critical section at any moment?

1

2

3

None of these

Solution-

Code for Processes P1, P2, … , P9

repeat

P(mutex)

{ Critical Section }

V(mutex)

forever

Code for Process P10

repeat

V(mutex)

{ Critical Section }

V(mutex)

forever

Consider mutex is initialized with 1.

Then-

All the 10 processes may be present inside the critical section at the same time.

If there would have been more number of processes, then they could also be present inside the critical section at the same time.

The explanation is as follows-

Explanation-

Scene-01:

Initially, process P1 arrives.

It performs the wait operation on mutex and sets its value to 0.

It then enters the critical section.

Scene-02:

Consider the processes P2, P3, … , P9 arrives when process P1 is executing the critical section.

All the other processes get blocked and are put to sleep in the waiting list.

Scene-03:

Process P10 arrives.

It performs the signal operation on mutex.

It selects one process from the waiting list say process P2 and wakes it up.

Process P2 can now enter the critical section (P1 is already present).

Now, process P10 enters the critical section (P1 and P2 are already present).

After executing critical section, during exit, it again performs the signal operation on mutex.

It selects one process from the waiting list say process P3 and wakes it up.

Process P3 can now enter the critical section (P1 and P2 are already present).

Process P10 may keep on executing repeatedly.

It wakes up 2 processes each time during its course of execution.

In this manner, all the processes blocked in the waiting list may get entry inside the critical section.

Thus, Option (D) is correct.